

Tech & Teach

Digital Chip Design Program

RTL Design

Task #13

Farah Alyahya

Rana Alsubaihi

Joud Almshari

Tala Alrsheed

Hoor Alkhedairi

Horiah Alqufaydi

Sandra Alomar

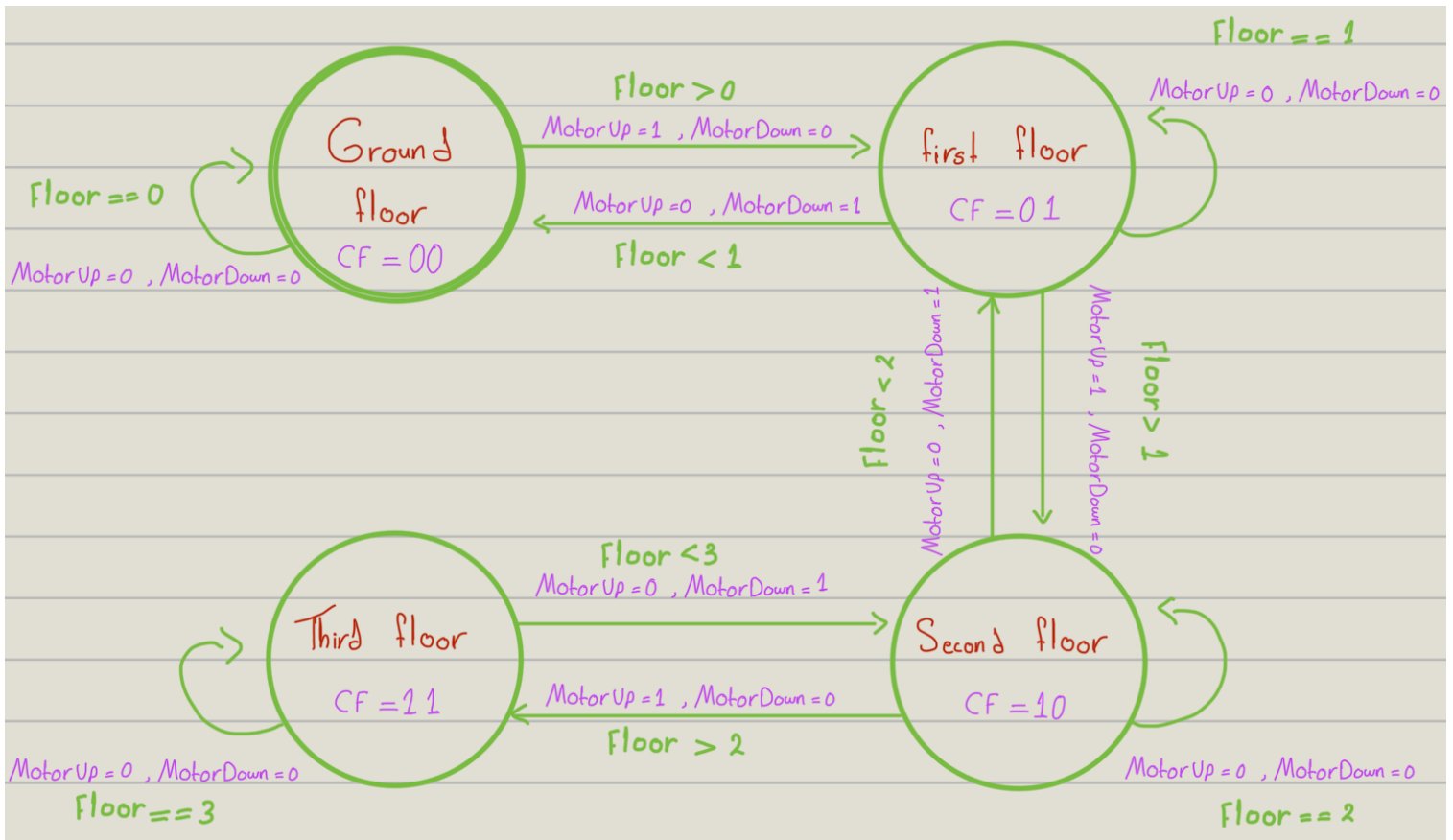
30/7/2026

Controller – Part 2

Version 1.0

Task #1.1

1.1.1



1.1.2

```
21 module ElevatorController (  
22     input  logic clk,  
23     input  logic rstn,  
24     input  logic [1:0] Floor,  
25     output logic [1:0] CurrentFloor,  
26     output logic MotorUp,  
27     output logic MotorDown  
28 );  
29  
30     enum logic [1:0] {  
31         Ground = 2'b00,  
32         First  = 2'b01,  
33         Second = 2'b10,  
34         Third  = 2'b11  
35     } current_state, next_state;  
36  
37     assign CurrentFloor = current_state;  
38  
39     always_comb begin  
40         next_state = current_state;  
41         MotorUp = 1'b0;  
42         MotorDown = 1'b0;  
43  
44         case (current_state)  
45             Ground: begin  
46                 if (Floor > 2'b00) begin  
47                     next_state = First;  
48                     MotorUp = 1'b1;  
49                 end  
50             end  
51  
52             First: begin  
53                 if (Floor > 2'b01) begin  
54                     next_state = Second;  
55                     MotorUp = 1'b1;  
56                 end  
57                 else if (Floor < 2'b01) begin
```

h

```

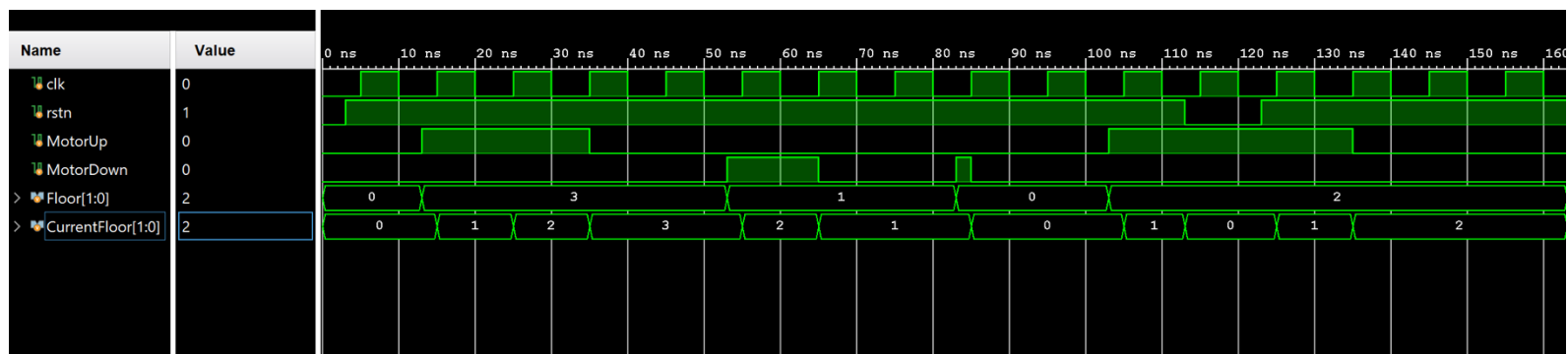
58         next_state = Ground;
59         MotorDown = 1'b1;
60     end
61 end
62
63 Second: begin
64     if (Floor > 2'b10) begin
65         next_state = Third;
66         MotorUp = 1'b1;
67     end
68     else if (Floor < 2'b10) begin
69         next_state = First;
70         MotorDown = 1'b1;
71     end
72 end
73
74 Third: begin
75     if (Floor < 2'b11) begin
76         next_state = Second;
77         MotorDown = 1'b1;
78     end
79 end
80
81 default: begin
82     next_state = Ground;
83 end
84 endcase
85 end
86
87 always @(posedge clk or negedge rstn) begin
88     if (!rstn)
89         current_state <= Ground;
90     else
91         current_state <= next_state;
92 end
93
94 endmodule
95
96

```

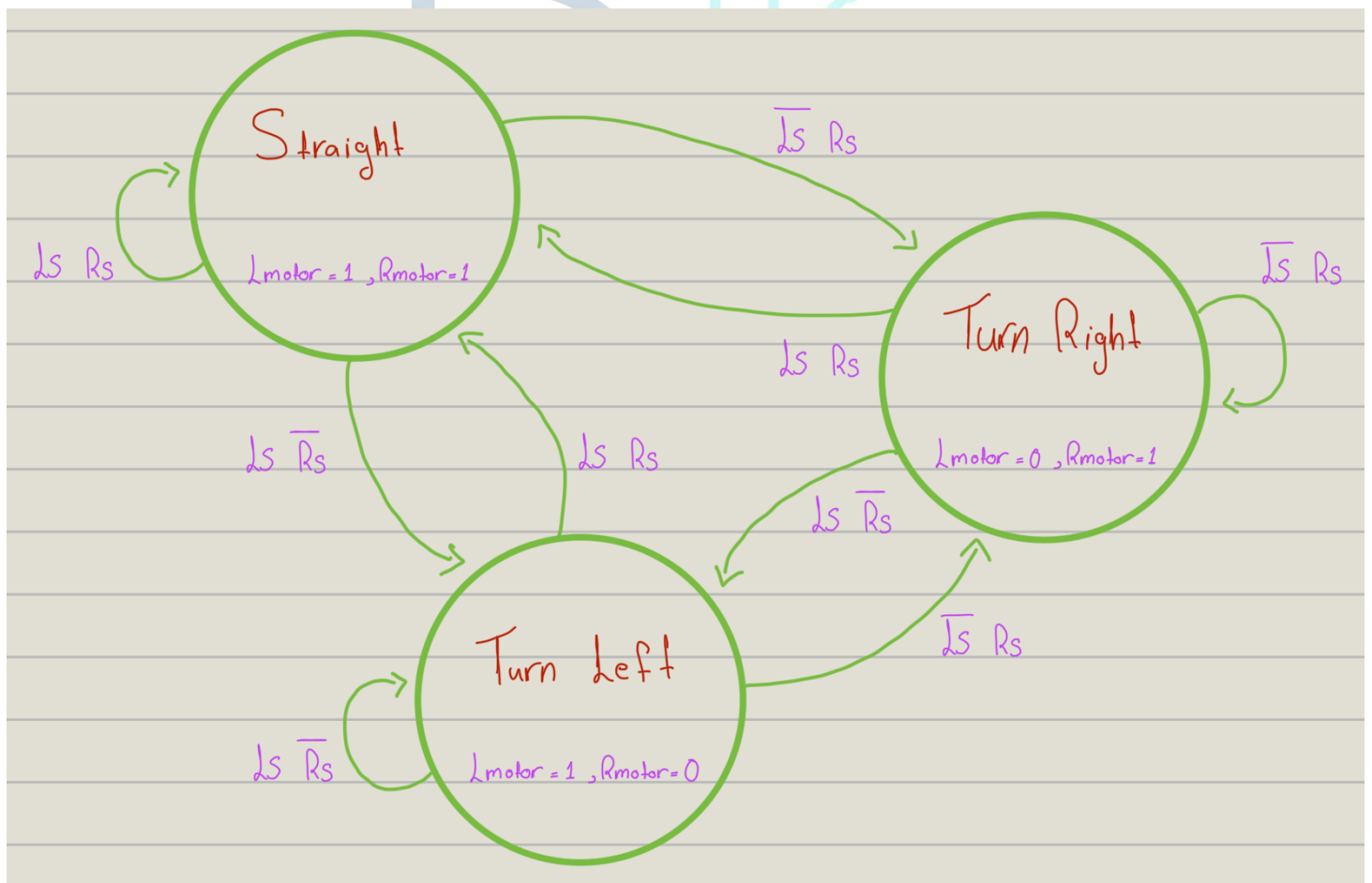
h

1.1.3

```
1 | `timescale 1ns / 1ps
2 | module ElevatorController_tb;
3 |
4 |     logic clk,rstn;
5 |     logic [1:0] Floor;
6 |     logic [1:0] CurrentFloor;
7 |     logic MotorUp,MotorDown;
8 |
9 |     ElevatorController dut (
10 |         .clk(clk),
11 |         .rstn(rstn),
12 |         .Floor(Floor),
13 |         .CurrentFloor(CurrentFloor),
14 |         .MotorUp(MotorUp),
15 |         .MotorDown(MotorDown)
16 |     );
17 |
18 |     always #5 clk = ~clk;
19 |
20 |     initial begin
21 |         clk = 0;
22 |         rstn = 0;
23 |         Floor = 2'b00;
24 |         #3 rstn = 1;
25 |
26 |         #10 Floor = 2'b11;
27 |         #40 Floor = 2'b01;
28 |         #30 Floor = 2'b00;
29 |         #20 Floor = 2'b10;
30 |
31 |         #10 rstn = 0;
32 |         #10 rstn = 1;
33 |         #40 $finish;
34 |     end
35 | endmodule
```



Optional task #X.1



Output logic :

```
1 module TrackingRobot(  
2     input logic clk,rstn,Lsensor,Rsensor,  
3     output logic Lmotor,Rmotor  
4 );  
5  
6     enum logic [1:0]{  
7         Straight,  
8         TurnR,  
9         TurnL,  
10        Lost =2'bxx  
11    } current_state, next_state;  
12  
13    always @(current_state)  
14        case(current_state)  
15            Straight : begin  
16                Lmotor = 1 ; Rmotor= 1;  
17            end  
18            TurnR : begin  
19                Lmotor = 0 ; Rmotor= 1;  
20            end  
21            TurnL : begin  
22                Lmotor = 1 ; Rmotor= 0;  
23            end  
24            default :begin  
25                Lmotor = 1'bxx ; Rmotor= 1'bxx;  
26            end  
27        endcase  
28
```

h

Next state logic :

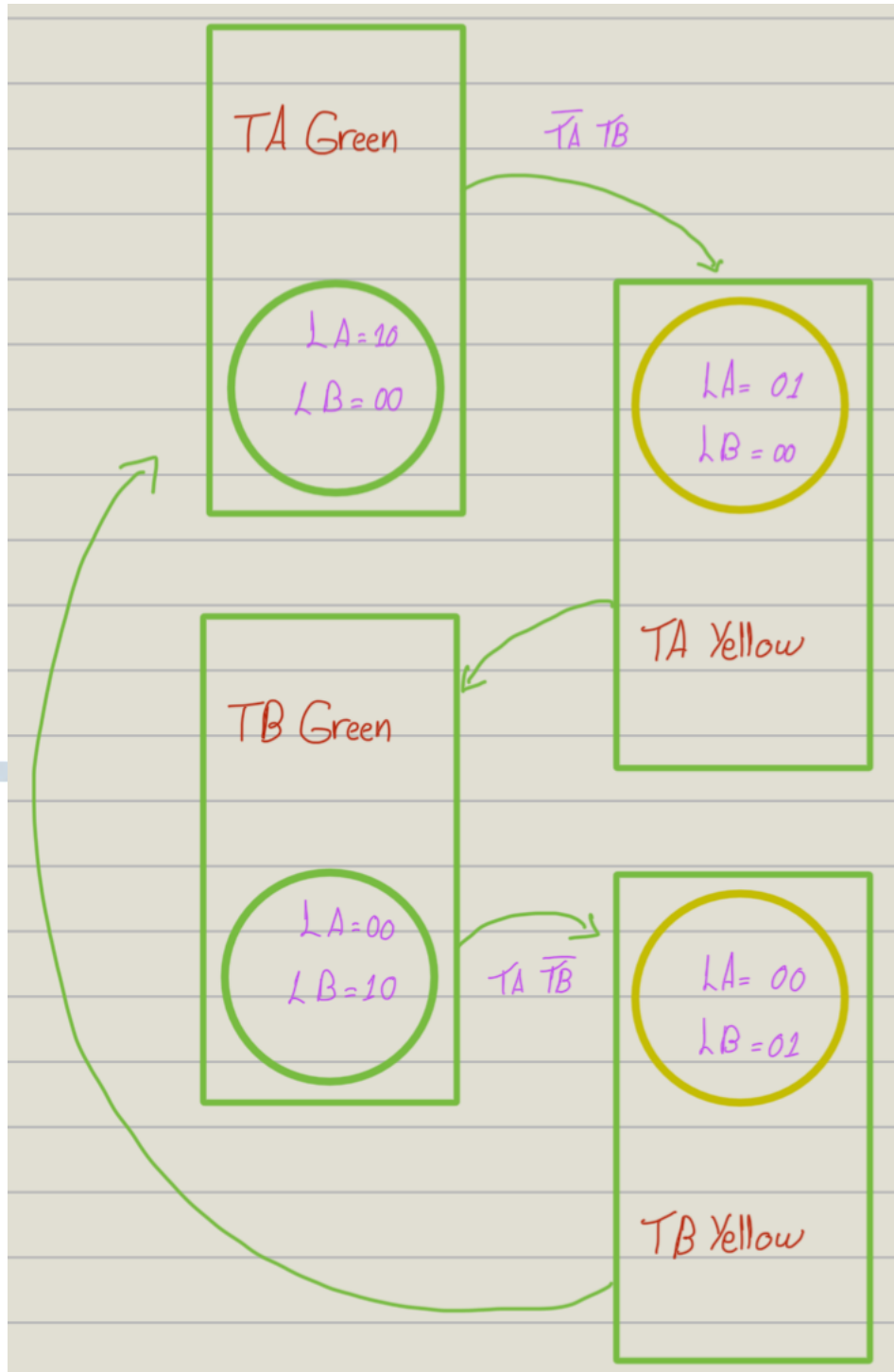
```

28 :
29 ⊞ always_comb begin
30 ⊞     case (current_state)
31 ⊞         Straight: begin
32 ⊞             if (Lsensor && Rsensor)
33 ⊞                 next_state = Straight;
34 ⊞             else if (Lsensor && !Rsensor)
35 ⊞                 next_state = TurnL;
36 ⊞             else if (!Lsensor && Rsensor)
37 ⊞                 next_state = TurnR;
38 ⊞             else
39 ⊞                 next_state = Lost;
40 ⊞         end
41 :
42 ⊞         TurnL: begin
43 ⊞             if (Lsensor && Rsensor)
44 ⊞                 next_state = Straight;
45 ⊞             else if (Lsensor && !Rsensor)
46 ⊞                 next_state = TurnL;
47 ⊞             else if (!Lsensor && Rsensor)
48 ⊞                 next_state = TurnR;
49 ⊞             else
50 ⊞                 next_state = Lost;
51 ⊞         end
52 :
53 ⊞         TurnR: begin
54 ⊞             if (Lsensor && Rsensor)
55 ⊞                 next_state = Straight;
56 ⊞             else if (Lsensor && !Rsensor)
57 ⊞                 next_state = TurnL;
58 ⊞             else if (!Lsensor && Rsensor)
59 ⊞                 next_state = TurnR;
60 ⊞             else
61 ⊞                 next_state = Lost;
62 ⊞         end
63 ⊞         default: begin
64 ⊞             next_state = Lost;
65 ⊞         end
66 ⊞     endcase
67 ⊞ end
68 :
69 :
70 ⊞ always @(posedge clk or negedge rstn)
71 ⊞     if (!rstn)
72 ⊞         current_state <= Straight;
73 ⊞     else
74 ⊞         current_state <= next_state;
75 :
76 ⊞ endmodule

```

h

Optional task #X.2



Output logic :

```
1 module TrafficLightController(  
2     input logic clk,rstn,TA,TB,  
3     output logic [1:0] LA,LB  
4 );  
5  
6     enum logic [1:0]{  
7         TAGreen,  
8         TAYellow,  
9         TBGreen,  
10        TBYellow,  
11        unused = 2'bxx  
12    }current_state, next_state;  
13  
14    always @(current_state)  
15        case(current_state)  
16            TAGreen : begin  
17                LA = 2'b10;  
18                LB = 2'b00;  
19            end  
20            TAYellow : begin  
21                LA = 2'b01;  
22                LB = 2'b00;  
23            end  
24            TBGreen : begin  
25                LA = 2'b00;  
26                LB = 2'b10;  
27            end  
28            TBYellow : begin  
29                LA = 2'b00;  
30                LB = 2'b01;  
31            end  
32            default : begin  
33                LA = 2'bxx;  
34                LB = 2'bxx;  
35            end  
36        endcase
```

ch

Next state logic :

```
38 ⊖ always_comb begin
39   | next_state = current_state;
40   ⊖ case (current_state)
41   ⊖   TAGreen: begin
42   ⊖     | if (!TA && TB)
43   |     | next_state = TAYellow;
44   |     | else
45   ⊖     | next_state = TAGreen;
46   ⊖   end
47   ⊖   TAYellow: begin
48   |     | next_state = TBGreen;
49   ⊖   end
50   ⊖   TBGreen: begin
51   ⊖     | if (TA && !TB)
52   |     | next_state = TBYellow;
53   |     | else
54   ⊖     | next_state = TBGreen;
55   ⊖   end
56   ⊖   TBYellow: begin
57   |     | next_state = TAGreen;
58   ⊖   end
59   ⊖   default: begin
60   |     | next_state = unused;
61   ⊖   end
62   ⊖ endcase
63   ⊖ end
64   ⊖ always @(posedge clk or negedge rstn)
65   ⊖   | if (!rstn)
66   |     | current_state <= TAGreen;
67   |     | else
68   ⊖     | current_state <= next_state;
69   ⊖ endmodule
```



Work distribution

Section	Farah	Joud	Tala	Horiah	Rana	Hoor	Sandra
Report	✓	✓	✓	✓	✓	✓	✓
Task 1.1	✓	✓	✓	✓	✓	✓	✓
Optional task #X.1						✓	
Optional task #X.2						✓	